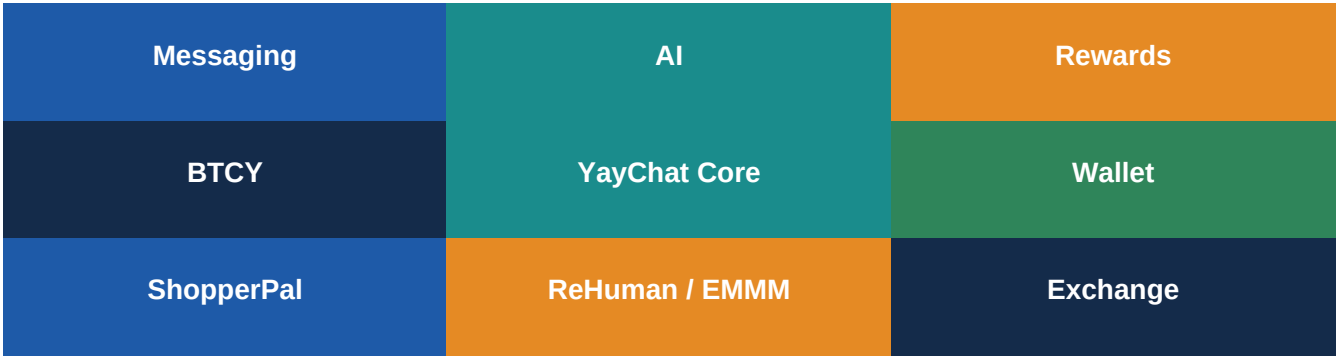


YAYCHAT

Platform Integration Architecture

How BTCY, ShopperPal, ReHuman, EMMM, Exchange, AI, Wallet, and future Indexx products connect through one shared platform



Version 1.0 | July 2026
Prepared for Indexx.ai

1. Executive Summary

A practical blueprint for connecting the entire Indexx ecosystem without creating one fragile monolithic system.

YayChat should act as the user-facing operating system of the Indexx ecosystem. Users see one app, one identity, one reward experience, and one wallet, while the underlying products remain independently maintainable and securely connected through standard APIs, events, permissions, and product adapters.

Core integration principle

One YayChat frontend, one shared identity, one API gateway, one reward engine, one reward ledger, one unified wallet, one event standard, separate product adapters, and separate product databases.

The recommended architecture does not merge BTCY, ShopperPal, ReHuman, EMMM, Exchange, AI, Wallet, messaging, and future products into a single codebase or database. Instead, YayChat coordinates them through a shared platform layer that provides consistent authentication, permissions, user experience, analytics, rewards, notifications, and security controls.

What this document defines

- The target platform architecture and service boundaries.
- How users authenticate once and access every connected product.
- How APIs, adapters, WebViews, deep links, and future mini apps should be used.
- How product actions become auditable events, rewards, wallet transactions, analytics, and notifications.
- How to keep product systems independent while delivering one unified user experience.
- A phased implementation roadmap and the decisions required before development begins.

2. Document Map

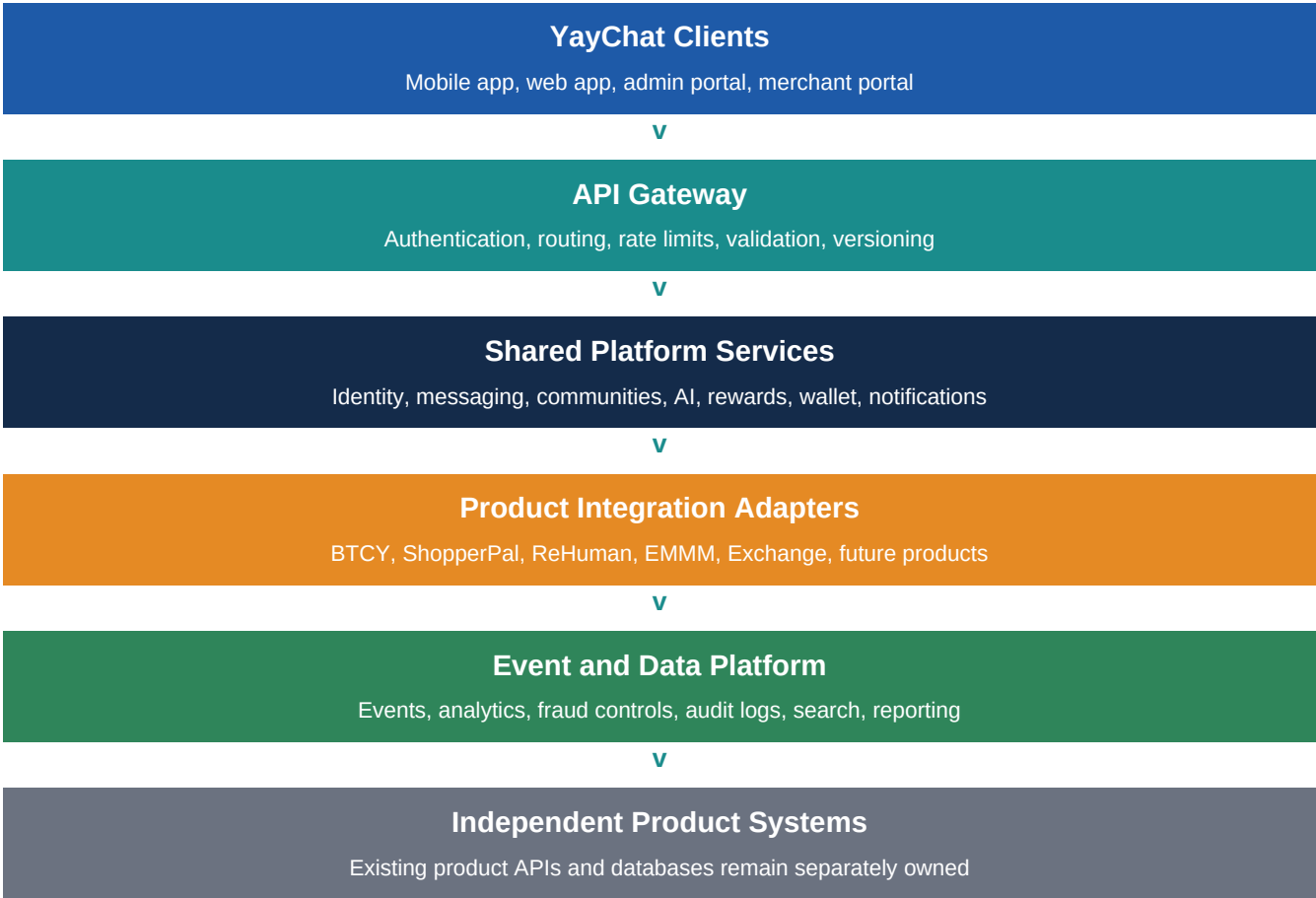
Section	Topic	Purpose
1	Executive Summary	The integration goal and recommended operating model
3	Target Architecture	Platform layers, ownership, and system boundaries
4	Shared Identity and Access	Single sign-on, account mapping, tokens, and permissions
5	API Gateway and Product Adapters	How the frontend communicates with independent products
6	Integration Methods	Native modules, WebViews, deep links, SDKs, and mini apps
7	Event-Driven Ecosystem	Standard events, analytics, notifications, and workflow orchestration
8	Rewards and Wallet Integration	Reward validation, ledgering, settlement, and transaction safety
9	Product-by-Product Connections	BTCY, ShopperPal, ReHuman, EMMM, Exchange, AI, and future apps
10	Data, Security, and Compliance	Isolation, privacy, financial controls, and operational safety
11	Implementation Roadmap	Phased delivery sequence and milestone outputs
12	API and Event Standards	Practical contracts for engineering teams
13	Decisions and Acceptance Criteria	What must be confirmed and how success is measured

Reading guidance

Product leaders can focus on Sections 3, 9, 11, and 13. Engineering teams should review the full document, especially Sections 4 through 8 and Section 12.

3. Target Architecture

A layered platform that looks unified to users but remains modular behind the scenes.



3.1 Responsibilities by layer

Layer	Primary Owner	Responsibilities
Client layer	Mobile, web, admin, merchant	Consistent navigation, design system, local state, secure token handling, offline behavior
API gateway	YayChat platform	Authentication, routing, throttling, validation, feature access, versioning, abuse protection
Shared platform services	YayChat platform	Identity, profiles, chat, communities, notifications, AI orchestration, rewards, wallet, search, moderation
Product adapters	Integration team and product owners	Translate product APIs and data into stable YayChat contracts
Event platform	Platform and data teams	Receive, validate, route, store, replay, and audit ecosystem events
Product systems	Individual product teams	Own product rules, source data, internal workflows, and product-specific compliance

3.2 Why not one codebase or database?

- A failure in one product should not take down every product.
- Wallet and financial records need stronger isolation than social content.
- Products may have different technology stacks, release cycles, legal requirements, and teams.
- Independent systems can scale and deploy separately while still sharing the same user experience.
- APIs and events create explicit contracts, ownership, auditability, and safer change management.

4. Shared Identity and Access

One account across the entire ecosystem, with controlled product-specific permissions.

Every user should receive a permanent YayChat user ID. Existing product accounts remain valid but are mapped to this central identity. A user signs in once, and YayChat issues short-lived, scoped access tokens for the products and services the user is allowed to access.

Example identity mapping

YayChat User ID: YC-100045
 BTCY account: BTCY-8832
 ShopperPal account: SP-5519
 ReHuman account: RH-2031
 EMMM account: EMMM-7114
 Exchange account: EX-9952

4.1 Authentication flow

Single sign-on and product access

Step	Owner	Action
1	User	Signs in to YayChat using an approved authentication method.
2	Identity service	Verifies credentials, device risk, account status, and multi-factor requirements.
3	Token service	Issues a short-lived YayChat access token and refresh token.
4	API gateway	Validates the token and checks requested permission scopes.
5	Product adapter	Exchanges or translates the YayChat token into a product-specific session when required.
6	Product API	Returns only the data permitted for that user and scope.

4.2 Required identity records

Field	Purpose
yaychat_user_id	Permanent ecosystem identity
product_name	BTCY, ShopperPal, ReHuman, EMMM, Exchange, or another product
external_user_id	The user ID in the product system
connection_status	Pending, active, restricted, suspended, or disconnected
permission_scopes	The approved access scopes for the product
verification_state	Identity, age, region, or KYC state where applicable
connected_at / updated_at	Audit timestamps for account linking changes

4.3 Example permission scopes

```

profile.read
wallet.balance.read
wallet.transfer.create
rewards.read
rewards.event.create
notifications.send
commerce.order.read
community.read
eligibility.age.read
eligibility.jurisdiction.read

```

Privacy rule

A product receives only the minimum information required for its function. Advertisers and merchants must never receive private chat content. AI access to chats, files, or history must be explicit, limited, and auditable.

5. API Gateway and Product Adapters

A single stable frontend contract, even when products use different technologies and data formats.

The YayChat mobile and web clients should communicate with one public API surface. The gateway routes each request to a shared platform service or a product adapter. The client should not contain product-specific authentication logic or direct database access.

```

YayChat App
|
v
YayChat API Gateway
|-- /api/v1/profile/*
|-- /api/v1/chat/*

```

```

|-- /api/v1/rewards/*
|-- /api/v1/wallet/*
|-- /api/v1/btcy/*
|-- /api/v1/shopperpal/*
|-- /api/v1/rehuman/*
|-- /api/v1/emmm/*
`-- /api/v1/exchange/*

```

5.1 Gateway responsibilities

- Authenticate requests and validate token signatures.
- Check permissions, product availability, region, age, and account state.
- Apply rate limits and block abusive traffic.
- Validate request and response schemas.
- Route traffic to the correct platform service or product adapter.
- Provide consistent errors, correlation IDs, metrics, and audit logs.
- Support API versioning, feature flags, canary releases, and gradual migration.

5.2 Adapter responsibilities

- Translate existing product fields into stable YayChat models.
- Handle product-specific authentication and token exchange.
- Normalize errors, pagination, currencies, statuses, and timestamps.
- Protect product systems from excessive traffic through caching and limits.
- Publish standard ecosystem events after verified product actions.
- Avoid embedding business rules that belong inside the source product.

Adapter normalization example

Product response:

```
{ "available_nuggets": 1400, "btcy_total": 22.5 }
```

Normalized YayChat response:

```

{
  "product": "btcy",
  "balances": [
    { "asset": "NUGGET", "amount": 1400 },
    { "asset": "BTCY", "amount": 22.5 }
  ]
}

```

6. Integration Methods

Use the simplest safe method for each product instead of forcing every integration into one technical pattern.

Method	Use When	Advantages	Risks / Limits
Native module	Core, high-frequency, sensitive experiences	Best UX, strong control, offline support	More development and maintenance
API-driven embedded screen	Products with mature APIs	Consistent YayChat UI with product-owned backend	Requires stable APIs and contracts

Method	Use When	Advantages	Risks / Limits
Secure WebView	Existing web products needing rapid integration	Fast launch and reuse of existing product	Limited native experience; token and navigation risks
Deep link	Temporary access to an existing app or website	Fastest option with low integration effort	Breaks the unified experience and may require another login
SDK	Reusable product capability or native device integration	Good performance and reusable integration	Versioning and mobile release coordination
Mini app	Later internal or third-party ecosystem expansion	Scalable distribution model	Requires sandboxing, review, permissions, and governance

6.1 Recommended initial choices

Capability	Recommended Initial Method
Messaging, profiles, communities, notifications	Native YayChat modules
AI assistant and AI inside conversations	Native experience with an AI orchestration API
Rewards Center and wallet summary	Native experience backed by shared services
BTCY balance and mining status	API-driven native screens; secure WebView only for unsupported flows
ShopperPal	API-driven product catalog and checkout where possible; WebView as a transition
ReHuman	API-driven discovery and bookings; WebView for complex legacy flows
EMMM	Restricted native or API-driven module after legal and regional controls are ready
Exchange	Limited portfolio and information first; regulated transactions only after compliance approval

Mini apps are not phase one

A mini-app platform should be introduced only after identity, permissions, wallet, events, review processes, sandboxing, analytics, and internal product integrations are proven.

7. Event-Driven Ecosystem

APIs serve immediate requests; events connect rewards, analytics, notifications, fraud detection, and cross-product workflows.

Every important verified action should produce a standard event. The source product owns confirmation that the action occurred. The event platform validates and routes the event to authorized consumers. Consumers do not directly modify the source product record.

Example event names

chat.message_sent


```
community.joined
ai.task_completed
btcy.mining_completed
shopperpal.order_completed
rehuman.booking_completed
emmm.prediction_settled
referral.user_verified
ad.rewarded_view_completed
wallet.transfer_completed
```

7.1 Standard event envelope

```
{
  "event_id": "evt_12345",
  "event_type": "shopperpal.order_completed",
  "event_version": 1,
  "user_id": "YC-100045",
  "source_product": "shopperpal",
  "source_reference": "ORDER-9821",
  "occurred_at": "2026-07-21T08:30:00Z",
  "idempotency_key": "shopperpal:ORDER-9821:completed",
  "metadata": { "order_total": 150, "currency": "USD" }
}
```

7.2 Event consumers

Consumer	Use of Event
Reward engine	Checks campaign eligibility, caps, fraud, and reward amount
Analytics	Records funnels, retention, transitions, revenue, and product adoption
Notification service	Sends relevant confirmations, reminders, and reward messages
Fraud and risk service	Scores behavior, devices, referrals, velocity, and abnormal activity
Wallet / ledger	Processes approved financial or reward entries through controlled commands
Campaign engine	Tracks campaign progress, limits, costs, and performance
Support and audit tools	Provides traceability for disputes and operational investigation

Important distinction

An event describes something that already happened. A command requests a controlled action, such as crediting a reward or initiating a wallet transfer. Financial services should never update balances directly from unvalidated event data.

8. Rewards and Wallet Integration

Centralize economic logic so every product follows the same controls, audit rules, and user experience.

8.1 Reward processing flow

Step	Owner	Action
1	Source product	Confirms a qualifying action and publishes a verified event.
2	Event platform	Validates schema, signature, idempotency key, product authority, and timestamps.
3	Reward engine	Evaluates campaign rules, user eligibility, limits, cooldowns, and fraud indicators.
4	Reward ledger	Records the approved, pending, rejected, reversed, or expired reward entry.
5	Wallet service	Credits a wallet asset only when the reward definition requires wallet settlement.
6	Notification service	Explains the reward, status, amount, conditions, and availability to the user.
7	Analytics and finance	Tracks cost, liability, campaign performance, reversals, and reconciliation.

8.2 Reward states

State	Meaning
Pending	Action recorded but validation, settlement, or waiting period is incomplete
Approved	Eligibility passed and the reward is authorized
Available	User may use or transfer the reward according to policy
Locked	Reward exists but is temporarily restricted
Rejected	The action did not meet eligibility or verification rules
Reversed	Previously issued value was invalidated or refunded
Expired	The reward passed its permitted usage period

8.3 Wallet boundaries

- Products request wallet operations through approved service APIs; they never write balances directly.
- The wallet uses double-entry or equivalent auditable accounting for money-like value.
- Available, pending, locked, and reversed balances remain separately traceable.
- Every transaction includes a unique reference, authorization context, actor, timestamps, and audit history.
- Wallet services are isolated from messaging and general product databases.

- High-risk transactions require stronger authentication, monitoring, limits, and recovery controls.

Economic decision required

The relationship among nuggets, BTCY, loyalty points, cashback, transferable assets, expiration, purchase, redemption, and cash-out must be formally defined before the reward and wallet services are finalized.

9. Product-by-Product Connections

A consistent integration contract, adapted to each product's maturity, risk, and user journey.

Product	Primary Functions	Shared Services	Recommended Connection
BTCY	Balance, mining, Alchemy, referrals, buy/sell entry points	Identity, rewards, wallet, notifications, analytics	API adapter plus native screens; restricted WebViews for legacy flows
ShopperPal	Products, coupons, merchants, checkout, orders, cashback	Identity, wallet, rewards, merchant service, ads, analytics	API-driven commerce; secure WebView as transition
ReHuman	Services, discovery, ordering, booking, support	Identity, wallet, rewards, chat support, notifications	API adapter; WebView for unsupported ordering flows
EMMM	Markets, predictions, results, settlement	Identity, age and region eligibility, wallet, rewards, risk	Restricted module after legal approval and product APIs
Exchange	Assets, portfolio, prices, buy/sell, transaction history	Identity, KYC state, wallet, risk, compliance, notifications	Information first; regulated transaction access later
AI	Questions, writing, summaries, translation, images, agents	Identity, usage limits, billing, files, chats, safety, analytics	Shared AI orchestration service with native entry points
Future apps	Product-specific actions and content	Identity, permissions, events, wallet, rewards, analytics	Adapter, WebView, SDK, or later mini app based on readiness

9.1 Example: ShopperPal purchase and reward

Step	Owner	Action
1	User	Opens ShopperPal inside YayChat.
2	YayChat	Validates identity and product access.
3	ShopperPal adapter	Loads products through the ShopperPal API and normalizes the response.
4	ShopperPal	Processes the order and confirms completion.
5	Event platform	Receives shopperpal.order_completed with an idempotency key.
6	Reward engine	Checks the active Shop-to-Earn campaign and anti-fraud rules.
7	Reward ledger / wallet	Records cashback, points, or BTCY and settles value as configured.
8	YayChat	Displays the reward and sends a notification.

Step	Owner	Action
9	Analytics	Records the cross-product journey from YayChat to purchase and reward.

9.2 Example: BTCY mining activity

Step	Owner	Action
1	User	Views mining status in a native YayChat screen.
2	BTCY adapter	Requests mining and balance data from the BTCY system.
3	BTCY	Returns the authoritative mining status and balance.
4	BTCY	Publishes btcy.mining_completed when a verified cycle finishes.
5	Reward and wallet services	Record any connected reward or asset movement under approved rules.
6	YayChat	Updates the balance view and sends a completion notification.

10. Data, Security, and Compliance

Shared user experience does not mean unrestricted data sharing.

10.1 Recommended data separation

Identity database
 Messaging and community storage
 Reward rules database and reward ledger
 Wallet and transaction ledger
 Media object storage
 Search index
 Event stream and event archive
 Analytics warehouse

Separate product databases:
 BTCY | ShopperPal | ReHuman | EMMM | Exchange

10.2 Security controls

Area	Required Controls
Identity	MFA, device management, risk scoring, session revocation, account recovery
API	TLS, signed tokens, scopes, request validation, rate limits, WAF,

Area	Required Controls
	replay protection
Events	Schema validation, producer authentication, idempotency, ordering rules, dead-letter handling
Wallet	Transaction authorization, isolated secrets, audit logs, reconciliation, fraud monitoring
WebViews	Short-lived tokens, allowlisted domains, controlled navigation, no token exposure to page scripts
AI	User consent, data minimization, model access controls, retention policies, safety monitoring
Operations	Central observability, alerts, incident response, backup, recovery, release and rollback plans

10.3 Compliance gates

- Jurisdiction and age eligibility before EMMM or regulated financial access.
- KYC, AML, sanctions, transaction monitoring, custody, and licensing decisions for wallet and exchange services.
- Privacy consent, data export, deletion, retention, and advertiser access restrictions.
- Merchant verification and fraud controls for commerce and promotions.
- Clear labeling and user controls for advertisements and rewarded content.
- Moderation, reporting, appeals, spam prevention, and harmful-content controls for social features.

Release gate

EMMM, exchange transactions, custody, transferable rewards, and certain wallet functions should remain disabled by feature flags until legal, compliance, security, operational, and regional requirements are approved.

11. Implementation Roadmap

Build the shared foundation first, then connect products in controlled phases.

Phase	Focus	Primary Outputs
Phase 0	Product and architecture definition	MVP, target users, identity, reward economics, custody, compliance, API inventory, data models, threat model
Phase 1	Complete YayChat frontend foundation	Design system, onboarding, authentication screens, profiles, chat, groups, communities, notifications, settings, moderation UI
Phase 2	Core backend and real-time services	Identity, messaging, media, notifications, presence, search, admin tools, analytics foundation
Phase 3	AI and communities	AI orchestration, chat integration, translation, summarization, usage limits, community roles and moderation
Phase 4	Rewards and BTCY	Event platform, reward rules, ledger, daily

Phase	Focus	Primary Outputs
		check-in, referrals, selected Chat-to-Earn rules, BTCY adapter
Phase 5	Unified wallet	Wallet accounts, supported assets, transfers, transaction history, security, reconciliation, compliance controls
Phase 6	Commerce and service products	ShopperPal and ReHuman adapters, purchase and booking events, reward and payment integration
Phase 7	Regulated products	EMMM and Exchange after jurisdiction, identity, risk, licensing, and operational controls are approved
Phase 8	Merchant, advertising, and mini apps	Merchant tools, sponsored content, rewarded ads, developer SDK/runtime, review and sandboxing

11.1 Integration order

1. Unified identity and authentication.
2. Messaging, profiles, notifications, and communities.
3. AI assistant and usage controls.
4. Event platform, reward engine, and reward ledger.
5. BTCY balance and mining integration.
6. Unified wallet and transaction controls.
7. ShopperPal and ReHuman.
8. EMMM and Exchange after compliance approval.
9. Merchant services, advertising, and later mini-app capabilities.

Frontend milestone

The first milestone may complete the full approved frontend and design system using mock services, but every screen should be built against documented API contracts so backend integration does not require redesigning the UI.

12. API and Event Standards

Stable contracts let independent teams build and release without tightly coupling their codebases.

12.1 API standards

Standard	Requirement
Versioning	/api/v1/... with a documented deprecation policy
Authentication	OAuth 2.0 / OpenID Connect compatible tokens with scopes and short expiration
Identifiers	Stable opaque IDs; never expose internal database assumptions
Time	UTC ISO 8601 timestamps with explicit timezone
Money and assets	Decimal-safe amounts, explicit currency or asset code, no floating-

Standard	Requirement
	point accounting
Pagination	Consistent cursor or page contract across adapters
Errors	Stable error code, human message, correlation ID, field details, and retry guidance
Idempotency	Required for orders, rewards, transfers, and other repeat-sensitive operations
Observability	Trace ID, product source, latency, status, and safe structured logs
Documentation	OpenAPI specification, examples, ownership, environments, and change history

12.2 Minimum integration contract for every product

- Named product owner and technical owner.
- Authentication and authorization method.
- Account-linking and external ID mapping rules.
- API specification, limits, environments, and availability targets.
- Supported events, schemas, signatures, retries, and idempotency rules.
- Permissions and user consent requirements.
- Data classification and retention rules.
- Error handling, support escalation, and incident ownership.
- Analytics events and success metrics.
- Versioning, rollout, rollback, and deprecation process.

12.3 Example normalized endpoints

```
GET /api/v1/btcy/balance
GET /api/v1/btcy/mining-status
GET /api/v1/shopperpal/products
POST /api/v1/shopperpal/orders
GET /api/v1/rehuman/services
POST /api/v1/rehuman/bookings
GET /api/v1/emmm/markets
POST /api/v1/emmm/predictions
GET /api/v1/exchange/portfolio
POST /api/v1/wallet/transfers
GET /api/v1/rewards/history
```

13. Required Decisions and Acceptance Criteria

Development should begin with explicit decisions, ownership, and measurable integration outcomes.

13.1 Decisions required before implementation

Decision Area	Question
MVP	Which product journeys and reward loops are included in the first production release?

Decision Area	Question
Identity	Will existing accounts be merged, linked, or migrated? What is the authoritative user record?
Rewards	What are nuggets, how do they relate to BTCY, and which rewards are transferable or expiring?
Wallet	Custodial or non-custodial? Which assets and regions launch first? Who manages keys and recovery?
Product readiness	Which products have production APIs, test environments, web apps, mobile SDKs, and event support?
Compliance	Which regions, age rules, identity checks, licenses, and restrictions apply to each feature?
Operations	Who owns incidents, support, service levels, reconciliation, monitoring, and rollback?
Mini apps	Are mini apps required later, and what permissions, sandboxing, review, and revenue model apply?

13.2 Minimum acceptance criteria

Capability	Acceptance Criterion
Single sign-on	A user signs in once and accesses an enabled product without another password prompt.
Authorization	Every request is denied unless the required scope, region, verification, and account state are valid.
Stable API	The frontend uses documented YayChat contracts, not direct product database fields.
Event reliability	Duplicate delivery does not duplicate rewards, orders, or financial entries.
Auditability	Support can trace a user action from product record through event, reward decision, wallet entry, and notification.
Data isolation	No product can directly read or modify another product's database or wallet balances.
Resilience	A product outage degrades only that product area and does not block chat or unrelated modules.
Observability	Errors, latency, event failures, reward failures, and wallet failures are measurable and alertable.
Security	Tokens, secrets, personal data, WebViews, APIs, and high-risk actions pass security review.
Compliance	Restricted features remain unavailable outside approved users, regions, and legal conditions.

14. Final Recommended Model

The architecture in one page.

User experience

Users interact with one YayChat application, one profile, one notification center, one Rewards Center, and one wallet experience.

Platform foundation

Identity, permissions, messaging, communities, AI orchestration, rewards, wallet, notifications, analytics, moderation, search, and admin tools are shared platform services.

Product independence

BTCY, ShopperPal, ReHuman, EMMM, Exchange, and future products retain separate systems, owners, databases, and business rules.

Connection mechanism

The API gateway, product adapters, standard events, scoped tokens, and shared contracts connect the ecosystem safely and consistently.

Economic layer

The reward engine decides eligibility; the reward ledger records decisions; the wallet settles approved assets or value. Products never directly alter shared balances.

Delivery sequence

Start with identity and communication, add AI and communities, then rewards and BTCY, then wallet, commerce, services, regulated modules, advertising, and mini apps.

YayChat succeeds when users experience the Indexx ecosystem as one connected environment while each underlying product remains secure, independently maintainable, auditable, and capable of evolving without breaking the rest of the platform.

One YayChat frontend
 + One shared identity
 + One API gateway
 + One event standard
 + One reward engine and ledger
 + One unified wallet experience
 + Separate product adapters and databases
 = One connected Indexx ecosystem